



NOVA Information Management School

Data Science and Machine Learning Group Project 2025/2026

Reach for Change

Part 1: Uncovering Donor Profiles Through Clustering

Student Number	Name	Email
20251702	Davide Corbo	20251702@novaims.unl.pt
20251865	Luis Alcaide	20251865@novaims.unl.pt
20221685	Fifi Nyarko-Mensah	20221685@novaims.unl.pt
20251105	Adewusi Kehinde Abiola	20251105@novaims.unl.pt

May 2026

```
import pandas as pd
```

```
df = pd.read_csv('https://raw.githubusercontent.com/iddadobroc/DSML_Group11_Clustering/refs/heads/main/donors_descriptive.csv')
df.head()
```

```
df.info()
```

```
df.shape
```

```
df.isna().sum()
```

```
df
```

```
df1=df.copy()
```

```
df2=df.copy()
df2.drop(columns=["WEALTH_RATING"], inplace = True)
```

```
df1
```

```
df2['CONTROL_NUMBER'].duplicated().sum()
```

```
df1.set_index("CONTROL_NUMBER", inplace = True)
```

```
df1
```

2. Data Preprocessing

2.1 Fix Invalid Placeholder Values

Several columns use non-standard strings to represent missing data: `SES` and `URBANICITY` encode unknowns as `'?'`, and `HOME_OWNER` uses `'U'` for unknown ownership status. These are replaced with `NaN` so they can be handled consistently during imputation.

First, check the unique values and the count of invalid entries in `SES`.

```
df2 = df1.copy()
# check uniques SES before cleaning
print('SES - unique values:', df2['SES'].unique())
print('SES - total "?"', (df2['SES'] == '?').sum())
print('SES - missing (NaN):', df2['SES'].isna().sum())
```

Replace '?' in SES with NaN.

```
import numpy as np
df2['SES'] = df2['SES'].replace('?', np.nan)
print('SES - unique values:', df2['SES'].unique())
print('SES - missing (NaN):', df2['SES'].isna().sum())
```

Same pattern for URBANICITY: check unique values and count '?' entries.

```
print('URBANICITY - unique values:', df2['URBANICITY'].unique())
print('URBANICITY - total "?:', (df2['URBANICITY'] == '?').sum())
print('URBANICITY - missing (NaN):', df2['URBANICITY'].isna().sum())
```

Replace '?' in URBANICITY with NaN.

```
df2['URBANICITY'] = df2['URBANICITY'].replace('?', np.nan)

print('URBANICITY - unique values:', df2['URBANICITY'].unique())
print('URBANICITY - missing (NaN):', df2['URBANICITY'].isna().sum())
```

HOME_OWNER uses 'U' for unknown ownership status — replace with NaN.

```
print('HOME_OWNER unique before:', df2['HOME_OWNER'].unique())
df2['HOME_OWNER'] = df2['HOME_OWNER'].replace('U', np.nan)
print('HOME_OWNER unique after:', df2['HOME_OWNER'].unique())
```

2.2 Correct Out-of-Range Numeric Values

PEP_STAR and RECENT_STAR_STATUS are binary flags that should only take values in [0, 1], and INCOME_GROUP is an ordinal score in [1, 7]. The dataset contains float values outside these valid ranges, likely from data entry or encoding errors. Each column is clipped to its valid range and rounded to an integer.

First, inspect the unique values in PEP_STAR.

```
df2['PEP_STAR'].unique()
```

```
print('\nPEP_STAR unique before:', sorted(df2['PEP_STAR'].dropna().unique()))
df2['PEP_STAR'] = df2['PEP_STAR'].clip(0, 1).round(0)
print('PEP_STAR unique after:', sorted(df2['PEP_STAR'].dropna().unique()))
```

RECENT_STAR_STATUS — same approach as PEP_STAR: clip to [0, 1] and round.

```
print('\nRECENT_STAR_STATUS range before: [{:.2f}, {:.2f}].format(
    df2['RECENT_STAR_STATUS'].min(), df2['RECENT_STAR_STATUS'].max())
df2['RECENT_STAR_STATUS'] = df2['RECENT_STAR_STATUS'].clip(0, 1).round()
print('RECENT_STAR_STATUS unique after:', sorted(df2['RECENT_STAR_STATUS'].dropna().unique()))
```

Verify RECENT_STAR_STATUS values after clamping.

```
df2['RECENT_STAR_STATUS'].unique()
```

INCOME_GROUP is an ordinal score [1–7]. Inspect the unique values before clamping.

```
df2['INCOME_GROUP'].unique()
```

```
print('\nINCOME_GROUP range before: [{:.2f}, {:.2f}].format(
    df2['INCOME_GROUP'].min(), df2['INCOME_GROUP'].max())
df2['INCOME_GROUP'] = df2['INCOME_GROUP'].clip(1, 7).round()
print('INCOME_GROUP unique after:', sorted(df2['INCOME_GROUP'].dropna().unique()))
```

2.3 Fix Negative Values

Several financial and count variables — such as gift amounts, promotion counts, and income estimates — cannot meaningfully be negative. Scan all numeric columns for negative entries, then clip the identified columns to 0.

```
num_cols_all = df2.select_dtypes(include='number').columns
cols_with_negatives = np.array([c for c in num_cols_all if (df2[c] < 0).sum() > 0])
```

```
print('Colunas com valores negativos:')
print(cols_with_negatives)
print()
for c in cols_with_negatives:
    print(f' {c}: {(df2[c] < 0).sum()} negatives (min={df2[c].min():.3f})')
```

Clip all identified columns to a minimum of 0.

```
non_negative_cols = [
    'CHILDREN', 'FILE_CARD_GIFT', 'FREQUENCY_STATUS_97NK',
    'LAST_GIFT_AMT', 'LIFETIME_CARD_PROM', 'LIFETIME_GIFT_AMOUNT',
    'LIFETIME_GIFT_COUNT', 'LIFETIME_MAX_GIFT_AMT', 'LIFETIME_MIN_GIFT_AMT',
    'LIFETIME_PROM', 'MEDIAN_HOME_VALUE', 'MEDIAN_HOUSEHOLD_INCOME',
    'MONTHS_SINCE_FIRST_GIFT', 'MONTHS_SINCE_LAST_PROM_RESP',
    'NUMBER_PROM_12', 'PCT_ATTRIBUTE1', 'PCT_ATTRIBUTE2', 'PCT_ATTRIBUTE3',
    'PCT_ATTRIBUTE4', 'PCT_OWNER_OCCUPIED', 'PER_CAPITA_INCOME',
    'RECENT_AVG_CARD_GIFT_AMT', 'RECENT_AVG_GIFT_AMT',
    'RECENT_CARD_RESPONSE_COUNT', 'RECENT_CARD_RESPONSE_PROP',
    'RECENT_RESPONSE_COUNT', 'RECENT_RESPONSE_PROP', 'MONTHS_SINCE_LAST_GIFT',
    'CARD_PROM_12'
]

for col in non_negative_cols:
    if col in df2.columns:
        df2[col] = df2[col].clip(lower=0)

# Verify no negatives remain
remaining = {c: (df2[c] < 0).sum() for c in df2.select_dtypes(include='number').columns if (df2[c] < 0).sum() > 0}
print('Negatives:', remaining if remaining else "")
```

2.4 Handle Missing Values

After correcting invalid placeholders and out-of-range entries, check how many true missing values remain in each column. The imputation strategy varies by variable type.

```
missing = df2.isna().sum()
missing = missing[missing > 0].sort_values(ascending=False)
print('Columns with missing values:')
print(missing.to_string())
```

2.4.1 Categorical Columns — Mode Imputation

`DONOR_GENDER`, `HOME_OWNER`, `REGENCY_STATUS_96NK`, `SES`, and `URBANICITY` all have a small proportion of missing values. Since these are categorical (nominal or ordinal), imputing with the most frequent value (mode) is the safest choice and avoids introducing arbitrary ordinal assumptions.

```
cat_cols = ['DONOR_GENDER', 'HOME_OWNER', 'REGENCY_STATUS_96NK', 'SES', 'URBANICITY']
for col in cat_cols:
    n_miss = df2[col].isna().sum()
    mode_val = df2[col].mode()[0]
    print(f'{col}: {n_miss} missing | mode = {mode_val!r}')
    print(f' {dict(df2[col].value_counts(dropna=False))}')
    print()
```

```
cat_cols = ['DONOR_GENDER', 'HOME_OWNER', 'REGENCY_STATUS_96NK', 'SES', 'URBANICITY']
for col in cat_cols:
    mode_val = df2[col].mode()[0]
    df2[col] = df2[col].fillna(mode_val)

print('Missing in categorical columns')
print(df2[cat_cols].isna().sum())
```

2.4.2 CHILDREN — Median Imputation

CHILDREN is an integer count of dependants. The median is preferred over the mean because the distribution is right-skewed. After filling, values are rounded and clamped to [0, 6] to ensure valid non-negative integers.

```
col = 'CHILDREN'
print(f'{col}: {df2[col].isna().sum()} missing ({df2[col].isna().mean()*100:.1f}%)')
print(f'Median: {df2[col].median()}')
print(f'\nValue counts:')
print(df2[col].value_counts(dropna=False).sort_index())
```

```
df2['CHILDREN'] = df2['CHILDREN'].fillna(df2['CHILDREN'].median()).round().clip(0, 6)
print('CHILDREN after imputation:')
print(df2['CHILDREN'].value_counts(dropna=False).sort_index())
print(f'Missing: {df2["CHILDREN"].isna().sum()}')
```

Start coding or [generate](#) with AI.

2.4.3 DONOR_AGE — Median Imputation

```
col = 'DONOR_AGE'
print(f'{col}: {df2[col].isna().sum()} missing ({df2[col].isna().mean()*100:.1f}%)')
print(f'Median: {df2[col].median()} | Mean: {df2[col].mean():.1f}')
print(f'Min: {df2[col].min()} | Max: {df2[col].max()}')
```

DONOR_AGE has 26.5% missing values. Age distributions are typically right-skewed, so the median is a more robust imputation choice than the mean.

```
df2['DONOR_AGE'] = df2['DONOR_AGE'].fillna(df2['DONOR_AGE'].median())
print(f'{col}: {df2[col].isna().sum()} missing ({df2[col].isna().mean()*100:.1f}%)')
```

2.4.4 INCOME_GROUP — Median Imputation

```
col = 'INCOME_GROUP'
print(f'{col}: {df2[col].isna().sum()} missing ({df2[col].isna().mean()*100:.1f}%)')
print(f'Median: {df2[col].median()}')
print(f'\nValue counts (incl. NaN):')
print(df2[col].value_counts(dropna=False).sort_index())
```

INCOME_GROUP is an ordinal score [1–7] with 24.2% missing values. Impute with the median, then round and clamp to [1, 7] to keep valid integer categories.

```
df2['INCOME_GROUP'] = df2['INCOME_GROUP'].fillna(df2['INCOME_GROUP'].median()).round().clip(1, 7)
print(df2[col].value_counts(dropna=False).sort_index())
```

2.5 Duplicate Check & Correlation-Based Feature Selection

Before analysing feature relationships, remove any duplicate rows or repeated index values to ensure each donor appears exactly once. Then compute the Pearson correlation matrix to identify pairs of features that carry redundant information.

```
print('Duplicated index (CONTROL_NUMBER):', df2.index.duplicated().sum())
print('Duplicated rows (all columns):', df2.duplicated().sum())
```

```
df2 = df2[~df2.index.duplicated(keep='first')]
df2 = df2.drop_duplicates()
print('Shape after duplicate removal:', df2.shape)
```

2.5.1 Pearson Correlation Matrix

Visualise pairwise linear correlations across all numeric features. Pairs with $|r| > 0.75$ are considered highly redundant — one variable from each pair will be dropped in the next step.

```
import seaborn as sns
import matplotlib.pyplot as plt

# usar o dataset mais avançado que existir
```

```

if 'df5' in globals():
    df_corr_base = df5.copy()
elif 'df4' in globals():
    df_corr_base = df4.copy()
else:
    df_corr_base = df2.copy()

# Pearson só funciona em colunas numéricas
corr_df = df_corr_base.select_dtypes(include='number')
pearson_corr = corr_df.corr(method='pearson')

print('Dataset usado para Pearson:', 'df5' if 'df5' in globals() else ('df4' if 'df4' in globals() else 'df2'))
print('Nº de variáveis no diagrama:', pearson_corr.shape[0])

plt.figure(figsize=(20, 14))
sns.heatmap(
    pearson_corr,
    cmap='coolwarm',
    center=0,
    vmin=-1, vmax=1,
    square=False,
    linewidths=0.2,
    annot=True,
    fmt='.2f',
    annot_kws={'size': 7}
)
plt.title('Diagrama de Correlação de Pearson (todas as variáveis numéricas)')
plt.tight_layout()
plt.show()

```

```

import numpy as np
import pandas as pd

thr = 0.75

if 'pearson_corr' not in globals():
    corr_df = df5.select_dtypes(include='number') if 'df5' in globals() else df2.select_dtypes(include='number')
    pearson_corr = corr_df.corr(method='pearson')

corr_abs = pearson_corr.abs()
upper = corr_abs.where(np.triu(np.ones(corr_abs.shape), k=1).astype(bool))

pairs = (
    upper.stack()
    .reset_index()
    .rename(columns={'level_0': 'var1', 'level_1': 'var2', 0: 'abs_r'})
)
pairs = pairs[pairs['abs_r'] > thr].sort_values('abs_r', ascending=False).reset_index(drop=True)

# adicionar r com sinal para análise
pairs['r'] = pairs.apply(lambda x: pearson_corr.loc[x['var1'], x['var2']], axis=1)
pairs = pairs[['var1', 'var2', 'r', 'abs_r']]

print(f'Pares com |r| > {thr}: {len(pairs)}')
display(pairs)

# sugestao automatica simples: remover var2 de cada par
if len(pairs) > 0:
    suggested_remove = pairs['var2'].drop_duplicates().tolist()
    print('\nSugestao (automatica): remover uma por par, começando por estas:')
    print(suggested_remove)

```

2.5.2 Drop Correlated Features

Drop correlated features ($|r| > 0.75$) — keep the more informative variable in each pair.

- `LIFETIME_PROM` ($r=0.88$ with `LIFETIME_CARD_PROM`) → drop `LIFETIME_PROM`, keep `LIFETIME_CARD_PROM`
- `LIFETIME_GIFT_COUNT` ($r=0.87$ with `FILE_CARD_GIFT`) → drop `LIFETIME_GIFT_COUNT`, keep `FILE_CARD_GIFT`
- `MONTHS_SINCE_FIRST_GIFT` ($r=0.84$ with `LIFETIME_CARD_PROM`) → drop `MONTHS_SINCE_FIRST_GIFT`
- `RECENT_CARD_RESPONSE_COUNT` ($r=0.79$ with `RECENT_RESPONSE_COUNT`) → drop card count, keep total count
- `LIFETIME_MAX_GIFT_AMT` ($r=0.76$ with `LAST_GIFT_AMT`) → drop lifetime max, keep recency signal
- `MEDIAN_HOUSEHOLD_INCOME` ($r=0.75$ with `PER_CAPITA_INCOME`) → drop median household, keep per capita

Also drop `WEALTH_RATING` — pre-computed external score, not meaningful for unsupervised profiling.

```

drop_cols = [
    'LIFETIME_PROM',          # r=0.88 with LIFETIME_CARD_PROM
    'LIFETIME_GIFT_COUNT',    # r=0.87 with FILE_CARD_GIFT
    'MONTHS_SINCE_FIRST_GIFT', # r=0.84 with LIFETIME_CARD_PROM
    'RECENT_CARD_RESPONSE_COUNT', # r=0.79 with RECENT_RESPONSE_COUNT
    'LIFETIME_MAX_GIFT_AMT',  # r=0.76 with LAST_GIFT_AMT

```

```

'MEDIAN_HOUSEHOLD_INCOME', # r=0.75 with PER_CAPITA_INCOME
'WEALTH_RATING',           # pre-computed external score
]

df_clean = df2.drop(columns=drop_cols, errors='ignore')

print(f'Dropped: {[c for c in drop_cols if c in df2.columns]}')
print(f'Shape before: {df2.shape}')
print(f'Shape after: {df_clean.shape}')

# Updated Pearson after feature reduction
corr_clean = df_clean.select_dtypes(include='number').corr(method='pearson')

plt.figure(figsize=(20, 14))
sns.heatmap(
    corr_clean,
    cmap='coolwarm',
    center=0,
    vmin=-1, vmax=1,
    linewidths=0.2,
    annot=True,
    fmt='.2f',
    annot_kws={'size': 7}
)
plt.title('Pearson - after dropping correlated features')
plt.tight_layout()
plt.show()

```

2.5.3 Final Check After Feature Reduction

Verify the shape of the reduced dataset, confirm there are no remaining missing values, and list any object columns that still need encoding.

```

print('Shape:', df_clean.shape)
print('Missing values:', df_clean.isna().sum().sum())
print('\nDtypes:')
print(df_clean.dtypes.value_counts())
print('\nObject columns still to encode:')
print(df_clean.select_dtypes(include='object').columns.tolist())

```

```

miss_detail = df_clean.isna().sum()
miss_detail = miss_detail[miss_detail > 0].sort_values(ascending=False)
print('Columns with missing values:')
print(miss_detail.to_string())

```

2.5.4 Remaining Numeric Columns — Median Imputation

After removing correlated features, a small number of numeric columns in `df_clean` still have missing values (~2% each). Impute with the median.

```

num_cols_missing = df_clean.select_dtypes(include='number').columns[df_clean.select_dtypes(include='number').isna().any()]

for col in num_cols_missing:
    median_val = df_clean[col].median()
    df_clean[col] = df_clean[col].fillna(median_val)

print(f'Imputed {len(num_cols_missing)} columns with median.')
print('Missing values remaining:', df_clean.isna().sum().sum())

```

2.6 Categorical Encoding

At this stage, the dataset is much cleaner, but we still have an important issue: clustering algorithms cannot work with categorical variables stored as text.

The columns `DONOR_GENDER`, `HOME_OWNER`, `REGENCY_STATUS_96NK`, `SES`, and `URBANICITY` still contain qualitative information. Leaving them as strings means the model cannot interpret them numerically. Using the wrong encoding could also create artificial distances between donors and lead to misleading clusters.

The problem at this point is twofold:

1. Convert categorical values into numeric form.
2. Do it in a way that preserves the meaning of each variable.

Approach

- `SES` and `URBANICITY` have a natural order, so ordinal encoding is appropriate.
- `REGENCY_STATUS_96NK` also reflects an ordered activity level, so we encode it ordinally as well.
- `DONOR_GENDER` is nominal, so we use one-hot encoding to avoid imposing a false ranking between categories.
- `HOME_OWNER` is dropped because it has zero variance after imputation (all donors have the same value), meaning it carries no useful information for separating profiles.

After this step, the dataset becomes fully numeric and suitable for distance-based clustering methods.

```
for col in ['DONOR_GENDER', 'HOME_OWNER', 'REGENCY_STATUS_96NK', 'SES', 'URBANICITY']:
    print(f' {col}: {sorted(df_clean[col].dropna().unique())}')
```

```
df_encoded = df_clean.copy()

# HOME_OWNER - zero variance (all 'H' after imputation), drop it
df_encoded.drop(columns=['HOME_OWNER'], inplace=True)

# SES - ordinal 1-4, stored as string → cast to int
df_encoded['SES'] = df_encoded['SES'].astype(int)

# URBANICITY - ordinal urban gradient: Urban > Suburban > City > Town > Rural
urbanicity_map = {'U': 5, 'S': 4, 'C': 3, 'T': 2, 'R': 1}
df_encoded['URBANICITY'] = df_encoded['URBANICITY'].map(urbanicity_map)

# REGENCY_STATUS_96NK - ordinal activity: Star > Active > First-time > New > Lapsing > Expired
recency_map = {'S': 5, 'A': 4, 'F': 3, 'N': 2, 'L': 1, 'E': 0}
df_encoded['REGENCY_STATUS_96NK'] = df_encoded['REGENCY_STATUS_96NK'].map(recency_map)

# DONOR_GENDER - nominal (F, M, U) → one-hot encode, drop 'F' as reference
gender_dummies = pd.get_dummies(df_encoded['DONOR_GENDER'], prefix='DONOR_GENDER', drop_first=True)
df_encoded = pd.concat([df_encoded.drop(columns=['DONOR_GENDER']), gender_dummies], axis=1)

print('Encoding complete.')
print('Shape:', df_encoded.shape)
print('Remaining object columns:', df_encoded.select_dtypes(include='object').columns.tolist())
print('New GENDER columns:', [c for c in df_encoded.columns if 'DONOR_GENDER' in c])
print('\nSample counts:')
print(gender_dummies.sum())
```

2.7 Feature Scaling - RobustScaler

After encoding the categorical variables, the dataset is now numeric, but there is still another methodological problem: the features are on very different scales.

For example, variables such as "LIFETIME_GIFT_AMOUNT" or "MEDIAN_HOME_VALUE" can take very large values, while proportions, binary indicators, or small counts operate on a much smaller scale. If clustering is applied directly at this point, the variables with the largest magnitudes will dominate the distance calculation, even if they are not the most informative for separating donor profiles.

In practice, that means the clustering algorithm could group donors mainly by numeric size rather than by meaningful behavioral similarity.

Current Approach

To solve this, scale the features using `RobustScaler` from `sklearn`. This is a good choice here because several donation-related variables are right-skewed and contain outliers. Unlike "StandardScaler", "RobustScaler" relies on the median and interquartile range, which makes it less sensitive to extreme values.

The goal of this step is not to change the meaning of the data, but to put the variables on a comparable scale. After scaling, each feature still represents the same concept, but with a more balanced contribution to the distance calculations used in clustering.

```
from sklearn.preprocessing import RobustScaler

scaler = RobustScaler()

df_scaled = pd.DataFrame(
    scaler.fit_transform(df_encoded),
    index=df_encoded.index,
    columns=df_encoded.columns
)

print('Scaled shape:', df_scaled.shape)
print('\nCentral tendencies after scaling, ~0 ?')
print(df_scaled.median().round(3).to_string())
```

`df_final` is now fully numeric, free of missing values, aligned with the assumptions of distance-based clustering methods.

```
df_final = df_scaled.copy()
for col in df_final.columns:
    print(f' {col}')
df_final.describe().round(3)
```

✧ Clustering Perspectives

To gain a comprehensive understanding of the donor base, we adopt two distinct clustering perspectives: a **behavioral perspective** and a **socio-economic perspective**. This approach allows us to analyze donors both in terms of their interaction patterns with the organization and their underlying demographic characteristics.

1. Behavioral Perspective

The behavioral perspective focuses on how donors interact with the organization, particularly in terms of their donation patterns and engagement levels. This approach is inspired by the well-known RFM (Recency, Frequency, Monetary) framework commonly used in marketing analytics.

Selected Features

- **RECENT_AVG_GIFT_AMT** → Average recent donation amount
- **RECENT_RESPONSE_PROP** → Engagement level with recent campaigns
- **RECENT_RESPONSE_COUNT** → Number of recent responses
- **MONTHS_SINCE_LAST_GIFT** → Recency of last donation
- **LIFETIME_GIFT_AMOUNT** → Total contribution over time
- **LAST_GIFT_AMT** → Most recent donation value

Objective

The goal of this perspective is to identify segments such as:

- High-value and loyal donors
- Occasionally engaged donors
- Inactive or lapsed donors

This segmentation is particularly useful for designing targeted fundraising strategies and improving donor retention.

2. Socio-Economic Perspective

The socio-economic perspective focuses on the characteristics of donors, such as their demographic profile, economic status, and living environment. Unlike the behavioral approach, this perspective aims to understand *who the donors are*, rather than *how they behave*.

Selected Features

- **DONOR_AGE** → Age of the donor
- **INCOME_GROUP** → Income classification
- **MEDIAN_HOME_VALUE** → Property value proxy
- **PCT_OWNER_OCCUPIED** → Housing stability indicator
- **SES** → Socio-economic segment
- **URBANICITY** → Geographic context (urban/rural)
- **CHILDREN** → Household composition

Objective

The goal of this perspective is to identify segments such as:

- High-income urban donors
- Lower-income rural donors
- Family-oriented donor groups

This segmentation helps tailor communication strategies and better align campaigns with donor profiles.

Summary

By combining these two perspectives, we obtain a more holistic view of the donor base:

- The **behavioral perspective** explains *how donors act*
- The **socio-economic perspective** explains *who donors are*

This dual approach enables more effective and personalized outreach strategies.

Let's start with the behavioral perspective

```
df_cluster1 = df_final[['RECENT_AVG_GIFT_AMT', 'RECENT_RESPONSE_PROP', 'RECENT_RESPONSE_COUNT', 'MONTHS_SINCE_LAST_GIFT', 'LIFETIME_GIFT
```

```
# Check missing values
print(df_cluster1.isna().sum())
```

```
# Basic stats
print(df_cluster1.describe())
```

```
# Quantile-based clipping (winsorization)
lower = df_cluster1.quantile(0.01)
upper = df_cluster1.quantile(0.99)

df_cluster1 = df_cluster1.clip(lower=lower, upper=upper, axis=1)
```

To mitigate the influence of extreme values, a quantile-based clipping approach was applied,

- capping values at the 1st and 99th percentiles. This method preserves the distribution of the data while limiting the impact of outliers on distance-based clustering.

```
df_cluster1.describe()
```

After outlier treatment using quantile-based clipping, the dataset exhibits stable distributions

- suitable for distance-based clustering. K-Means is then applied, and the optimal number of clusters is determined using both the Elbow method and Silhouette analysis.

```
X = df_cluster1.values
```

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

ks = range(1, 11)
inertias = []

for k in ks:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X)
    inertias.append(kmeans.inertia_)

plt.figure(figsize=(8,5))
plt.plot(ks, inertias, marker='o')
plt.xlabel('number of clusters')
plt.ylabel('inertia')
plt.title('Elbow Method')
plt.xticks(ks)
plt.grid(True)
plt.show()
```

```
from sklearn.metrics import silhouette_score

sil_scores = []

for k in range(2, 11):
    kmeans = KMeans(n_clusters=k, random_state=42)
    labels = kmeans.fit_predict(X)
    score = silhouette_score(X, labels)
    sil_scores.append(score)

plt.plot(range(2,11), sil_scores, marker='o')
plt.xlabel('number of clusters')
plt.ylabel('silhouette score')
plt.title('Silhouette Scores')
plt.show()
```

```
from scipy.cluster import hierarchy

df_cluster1_sample = df_cluster1.sample(n = 1000, random_state = 100)

# create clusters using the "ward method"
clusters = hierarchy.linkage(df_cluster1_sample, method="ward")
```

```
# create the dendrogram using the hierarchy.dendrogram() method.
plt.figure(figsize=(8, 6))
dendrogram = hierarchy.dendrogram(clusters)
```

Methods	suggested K
Elbow	~5
Silhouette	3
Dendrogram	2-4

```
# =====
# K = 2
# =====
df_k2 = df_cluster1.copy()
kmeans2 = KMeans(n_clusters=2, random_state=42)
df_k2["cluster"] = kmeans2.fit_predict(X)

summary_k2 = df_k2.groupby("cluster").mean()

# =====
# K = 3
# =====
df_k3 = df_cluster1.copy()
kmeans3 = KMeans(n_clusters=3, random_state=42)
df_k3["cluster"] = kmeans3.fit_predict(X)

summary_k3 = df_k3.groupby("cluster").mean()

# =====
# K = 4
# =====
df_k4 = df_cluster1.copy()
kmeans4 = KMeans(n_clusters=4, random_state=42)
df_k4["cluster"] = kmeans4.fit_predict(X)

summary_k4 = df_k4.groupby("cluster").mean()

# =====
# K = 5
# =====
df_k5 = df_cluster1.copy()
kmeans5 = KMeans(n_clusters=5, random_state=42)
df_k5["cluster"] = kmeans5.fit_predict(X)

summary_k5 = df_k5.groupby("cluster").mean()
```

```
print("==== K = 2 ====")
print(summary_k2)

print("\n==== K = 3 ====")
print(summary_k3)

print("\n==== K = 4 ====")
print(summary_k4)

print("\n==== K = 5 ====")
print(summary_k5)
```

```
# Cluster distribution
print("\nCluster sizes (k=2):")
print(df_k2["cluster"].value_counts(normalize=True))

print("\nCluster sizes (k=3):")
print(df_k3["cluster"].value_counts(normalize=True))

print("\nCluster sizes (k=4):")
print(df_k4["cluster"].value_counts(normalize=True))

print("\nCluster sizes (k=5):")
print(df_k5["cluster"].value_counts(normalize=True))
```

Final Selection of k

The clustering solutions for $k = 2$ to $K = 5$ were evaluated using both quantitative metrics and qualitative interpretation.

While $k = 2$ provides a coarse segmentation, it fails to capture meaningful behavioral differences. Increasing the number of clusters beyond $k = 3$ leads to over-segmentation, as observed in $k = 4$ and $k = 5$, where existing clusters are artificially split without introducing

new, distinct behavioral patterns.

In particular, $k = 5$ produces very small clusters (below 5% of the population) and unstable segments driven by extreme values, indicating a loss of robustness.

The solution with $k = 3$ provides the best balance between:

- statistical quality (highest silhouette score),
- cluster stability (no excessively small groups),
- and interpretability (clear and distinct donor profiles).

Therefore, **$k = 3$ is selected as the optimal number of clusters** for the behavioral perspective.

```
df_k3.groupby(['cluster']).describe().transpose()
```

✓ Cluster Profiling (Behavioral Perspective – $k = 3$)

Based on the descriptive statistics of each cluster, three distinct donor profiles emerge, characterized by clear differences in engagement behavior and monetary contribution.

Cluster 0 – Active Low-Value Donors ($\approx 31.6\%$)

This segment is defined by **high engagement but low monetary contribution**.

- **High responsiveness:**
 - RECENT_RESPONSE_PROP (mean ≈ 0.92)
 - RECENT_RESPONSE_COUNT (mean ≈ 1.10)
- **Recent activity:**
 - MONTHS_SINCE_LAST_GIFT is below average (mean ≈ -0.41)
- **Low donation values:**
 - RECENT_AVG_GIFT_AMT (mean ≈ -0.49)
 - LAST_GIFT_AMT (mean ≈ -0.54)

These donors are highly responsive and actively engaged with campaigns, but their contributions remain relatively small. They represent a **loyal but low-value segment**, potentially suitable for upselling strategies.

Cluster 1 – Low Engagement / Average Donors ($\approx 57.9\%$)

This is the **largest segment**, characterized by relatively low engagement and average donation behavior.

- **Low responsiveness:**
 - RECENT_RESPONSE_PROP (mean ≈ -0.20)
 - RECENT_RESPONSE_COUNT (mean ≈ -0.54)
- **Less recent activity:**
 - MONTHS_SINCE_LAST_GIFT is above average (mean ≈ 0.33)
- **Moderate donation levels:**
 - RECENT_AVG_GIFT_AMT and LAST_GIFT_AMT slightly above average

This group represents a **less engaged and partially inactive segment**, with no strong behavioral signals. They may require reactivation strategies or more personalized communication to increase engagement.

Cluster 2 – High-Value Donors ($\approx 10.6\%$)

This segment includes donors with **significantly higher monetary contributions**, both recently and over their lifetime.

- **High donation amounts:**
 - RECENT_AVG_GIFT_AMT (mean ≈ 1.51)
 - LAST_GIFT_AMT (mean ≈ 1.68)
 - LIFETIME_GIFT_AMOUNT (mean ≈ 1.85)
- **Moderate engagement:**
 - RECENT_RESPONSE_PROP and RECENT_RESPONSE_COUNT around average
- **Relatively recent activity:**
 - MONTHS_SINCE_LAST_GIFT slightly below average

These donors are the **most valuable segment in terms of revenue**, although they are not the most responsive. They should be treated as priority individuals, with tailored retention and relationship-building strategies.

Key Insight

The clustering reveals a clear **trade-off between engagement and monetary value**:

- Highly engaged donors tend to contribute smaller amounts
- High-value donors are less responsive but financially significant
- A large portion of the population remains weakly engaged

This segmentation provides a strong foundation for designing differentiated outreach strategies aligned with each donor profile.

```
from sklearn.metrics.pairwise import euclidean_distances

dists = euclidean_distances(kmeans3.cluster_centers_)
dists
```

Distance Between Cluster Centroids

To further evaluate the separation between clusters, Euclidean distances between cluster centroids were computed.

The results show that the largest distance is observed between Cluster 0 and Cluster 2, indicating that these two groups represent the most distinct behavioral profiles. In particular, Cluster 0 is characterized by high engagement and low donation amounts, while Cluster 2 represents high-value donors.

Cluster 1 appears to occupy an intermediate position, as it shows moderate distances from both Cluster 0 and Cluster 2. This suggests that it acts as a transitional segment between low-value engaged donors and high-value contributors.

Overall, the centroid distances confirm that the clustering solution is well-structured, with clearly differentiated segments and meaningful behavioral separation.

```
centroids_k3 = pd.DataFrame(kmeans3.cluster_centers_, columns=df_cluster1.columns)
centroids_k3
```

```
# Import PCA
from sklearn.decomposition import PCA
import pandas as pd

# Select features only (exclude cluster column)
X_features = df_k3.drop(columns="cluster")

# Apply PCA with 2 components
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_features)

# Create PCA DataFrame
df_pca = pd.DataFrame(X_pca, columns=["PC1", "PC2"])
df_pca["cluster"] = df_k3["cluster"].values
```

```
# Create figure
plt.figure(figsize=(8,6))

# Scatter plot (points)
scatter = plt.scatter(
    df_pca["PC1"],
    df_pca["PC2"],
    c=df_pca["cluster"],
    cmap="viridis",
    s=10,
    alpha=0.5
)

# Transform centroids into PCA space
centroids_pca = pca.transform(kmeans3.cluster_centers_)

# Plot centroids
plt.scatter(
    centroids_pca[:, 0],
    centroids_pca[:, 1],
    marker='X',
    s=200,
    color='black',
    label='Centroids'
)

# Create legend for clusters
```

```
legend1 = plt.legend(*scatter.legend_elements(), title="Clusters")

# Add centroid legend
plt.gca().add_artist(legend1)
plt.legend()

# Labels and title
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("PCA Projection of Clusters with Centroids")

# Show plot
plt.show()
```

```
df_demog = df_clean[
    ["DONOR_GENDER", "SES", "URBANICITY", "INCOME_GROUP", "HOME_OWNER"]
].copy()

demo_view = pd.concat([df_k3, df_demog], axis=1)
```

```
df_demog["cluster"] = df_k3["cluster"].values
```

```
cluster0 = df_demog[df_demog["cluster"] == 0]
cluster1 = df_demog[df_demog["cluster"] == 1]
cluster2 = df_demog[df_demog["cluster"] == 2]
```

```
# Example: gender distribution
(cluster0["DONOR_GENDER"].value_counts(normalize=True)*100).round(2)
```

```
# Example: SES
(cluster1["SES"].value_counts(normalize=True)*100).round(2)
```

```
# Example: income
cluster2["INCOME_GROUP"].mean()
```

Cluster Profiling - Behavioral Segmentation (Demographic Enrichment)

To complement the behavioral clustering, demographic and socio-economic variables were analyzed across the three clusters. The results reveal subtle but meaningful differences that help refine the interpretation of each segment.

General Patterns

- Gender distribution is relatively consistent across all clusters, with a majority of female donors (about 55%) and a stable male representation (about 38 - 44%).
- All individuals are homeowners, indicating a homogeneous characteristic across the dataset that does not contribute to cluster differentiation.
- The majority of donors across all clusters belong to SES levels 1 and 2, suggesting a concentration in lower to middle socio-economic segments.

Cluster-Specific Insights

Cluster 0 — High Engagement, Lower Value

- Slightly higher concentration in SES level 2 (54.41%) with lower presence in SES level 1 (25.99%).
- More balanced urban distribution, with notable presence in suburban (S) and rural (R) areas.
- Lower average income (3.76), confirming that engagement is not necessarily driven by higher economic capacity.

Interpretation: This cluster consists of **highly engaged donors with moderate socio-economic profiles**, suggesting strong behavioral loyalty rather than financial strength.

Cluster 1 — Low Engagement, Mid Profile

- Similar SES distribution to Cluster 0, but slightly higher representation in SES level 1 (31.33%).
- Slightly higher presence in suburban areas.
- متوسط income level (4.00), indicating a middle-ground economic position.

Interpretation: This group represents **less engaged donors with average socio-economic characteristics**, potentially requiring reactivation strategies.

Cluster 2 — High Value Donors

- Highest proportion of individuals in SES level 1 (36.31%), indicating relatively stronger socio-economic positioning.
- Slightly higher male representation compared to other clusters.
- Highest average income (4.14), confirming economic capacity.
- More concentrated in suburban (S) areas.

Interpretation: This cluster includes **high-value donors with stronger economic profiles**, whose contribution is driven more by financial capability than engagement frequency.

Key Takeaways

- Donation behavior is not solely determined by socio-economic status: highly engaged donors are not necessarily the wealthiest.
- High-value donors tend to exhibit better economic indicators but not necessarily higher engagement.
- The clusters reflect a clear distinction between **engagement-driven** and **value-driven** donor profiles.

Overall, integrating behavioral and demographic perspectives provides a more comprehensive segmentation, enabling more targeted and effective strategic decisions.

Now we will proceed with the Socio-Economic perspective for the second clustering

```
df_final.columns
```

```
features_demo = [
    "DONOR_AGE",
    "CHILDREN",
    "INCOME_GROUP",
    "MEDIAN_HOME_VALUE",
    "PCT_OWNER_OCCUPIED",
    "SES",
    "URBANICITY"
]

df_cluster2 = df_final[features_demo].copy()
```

```
df_cluster2.describe()
```

```
# Apply quantile clipping to reduce extreme values
for col in df_cluster2.columns:
    lower = df_cluster2[col].quantile(0.01)
    upper = df_cluster2[col].quantile(0.99)
    df_cluster2[col] = df_cluster2[col].clip(lower, upper)
```

```
df_cluster2.describe()
```

Outlier Treatment in Demographic Features

The descriptive analysis revealed the presence of moderate outliers, particularly in variables such as median home value and housing-related indicators. These extreme values could disproportionately influence the clustering results, leading to segments driven primarily by a single dimension.

To mitigate this issue, a quantile-based clipping approach was applied, limiting the values to the 1st and 99th percentiles. This ensures a more balanced representation of the data and allows the clustering algorithm to capture broader socio-demographic patterns rather than being dominated by extreme observations.

```
df_cluster2["MEDIAN_HOME_VALUE"] = df_cluster2["MEDIAN_HOME_VALUE"].clip(
    df_cluster2["MEDIAN_HOME_VALUE"].quantile(0.01),
    df_cluster2["MEDIAN_HOME_VALUE"].quantile(0.99)
)
```

```
df_cluster2.describe()
```

```
df_cluster2.std().sort_values(ascending=False)
```

```
df_cluster2.corr()["MEDIAN_HOME_VALUE"].sort_values(ascending=False)
```

```
kmeans_tmp = KMeans(n_clusters=3, random_state=42).fit(df_cluster2)
centroids_tmp = pd.DataFrame(kmeans_tmp.cluster_centers_, columns=df_cluster2.columns)
centroids_tmp
```

```
import numpy as np

col = "MEDIAN_HOME_VALUE"

shift = abs(df_cluster2[col].min()) + 1
df_cluster2[col] = np.log1p(df_cluster2[col] + shift)
```

```
# Basic stats
print(df_cluster2.describe())

# Std ranking (to verify balance)
print("\nStd by feature:")
print(df_cluster2.std().sort_values(ascending=False))
```

```
corr = df_cluster2.corr()
print(corr["MEDIAN_HOME_VALUE"].sort_values(ascending=False))
```

✓ Feature Transformation Validation

The application of a logarithmic transformation to the MEDIAN_HOME_VALUE variable successfully reduced its variance and mitigated its dominant effect in the clustering process.

Post-transformation analysis shows that all variables now exhibit a balanced scale, with no single feature disproportionately influencing the results. Additionally, the correlation structure confirms that the variable retains its informational value without introducing redundancy.

This ensures that the clustering algorithm captures a multidimensional socio-demographic structure, rather than being driven primarily by wealth-related characteristics.

```
X = df_cluster2.values
```

```
ks = range(1, 11)
inertias = []

for k in ks:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X)
    inertias.append(kmeans.inertia_)

plt.figure(figsize=(8,5))
plt.plot(ks, inertias, marker='o')
plt.xlabel('number of clusters')
plt.ylabel('inertia')
plt.title('Elbow Method')
plt.xticks(ks)
plt.grid(True)
plt.show()
```

```
sil_scores = []

for k in range(2, 11):
    kmeans = KMeans(n_clusters=k, random_state=42)
    labels = kmeans.fit_predict(X)
    score = silhouette_score(X, labels)
    sil_scores.append(score)

plt.plot(range(2,11), sil_scores, marker='o')
plt.xlabel('number of clusters')
plt.ylabel('silhouette score')
plt.title('Silhouette Scores')
plt.show()
```

```
df_cluster2_sample = df_cluster2.sample(n = 1000, random_state = 100)

# create clusters using the "ward method"
clusters = hierarchy.linkage(df_cluster2_sample, method="ward")

# create the dendrogram using the hierarchy.dendrogram() method.
plt.figure(figsize=(8, 6))
dendrogram = hierarchy.dendrogram(clusters)
```

Selection of the Optimal Number of Clusters

The optimal number of clusters was determined using a combination of the Elbow Method, Silhouette Analysis, and hierarchical clustering (dendrogram).

The Elbow Method suggests a clear inflection point at $k = 3$, where the reduction in inertia begins to diminish significantly. This is further supported by the Silhouette Score, which reaches its maximum value at $k = 3$, indicating the best balance between cluster cohesion and separation.

Additionally, the dendrogram reveals a natural grouping structure consistent with three main clusters, with subsequent splits appearing less significant.

Although the silhouette values are moderate, this is expected given the nature of demographic data, which tends to exhibit gradual transitions rather than sharply defined segments.

Overall, $k = 3$ was selected as the optimal number of clusters, providing a robust and interpretable segmentation of the dataset.

```
df_k3 = df_cluster2.copy()
kmeans3 = KMeans(n_clusters=3, random_state=42)
df_k3["cluster"] = kmeans3.fit_predict(X)

summary_k3 = df_k3.groupby("cluster").mean()
summary_k3
```

```
print("\nCluster sizes (k=3):")
df_k3["cluster"].value_counts(normalize=True)
```

```
df_k3.groupby(['cluster']).describe().transpose()
```

Demographic Cluster Interpretation

The demographic clustering reveals three distinct socio-economic profiles, driven primarily by income, housing characteristics, and geographic context.

Cluster 0 represents older individuals with lower income levels, typically residing in less urbanized areas. Despite their lower economic status, they exhibit relatively stable housing conditions.

Cluster 1 consists of younger and more affluent individuals, characterized by higher income levels, greater home ownership, and a stronger presence in urban environments.

Cluster 2 represents a more vulnerable segment, defined by significantly lower levels of home ownership. Although not the lowest in income, this group appears to experience reduced housing stability and is more concentrated in urban areas.

Overall, the segmentation highlights that housing conditions and geographic factors play a crucial role alongside income in defining socio-demographic profiles.

```
dists = euclidean_distances(kmeans3.cluster_centers_)
dists
```

```
centroids_k3 = pd.DataFrame(kmeans3.cluster_centers_, columns=df_cluster2.columns)
centroids_k3
```

Cluster Separation Analysis

The Euclidean distance between cluster centroids reveals a clear structure in the segmentation. The largest distance is observed between Cluster 1 and Cluster 2, indicating that these groups are the most dissimilar.

Cluster 2 stands out as a distinct segment primarily characterized by significantly lower levels of home ownership, suggesting reduced housing stability. In contrast, Clusters 0 and 1 appear closer to each other and represent more stable populations, differentiated mainly by income and socio-economic status.

This suggests a two-level segmentation structure: first, a separation based on housing stability, and second, a division based on economic conditions. Such a structure highlights the importance of non-income variables, particularly housing-related indicators, in defining meaningful demographic segments.

```
kmeans_demo = KMeans(n_clusters=3, random_state=42)

df_cluster2["cluster"] = kmeans_demo.fit_predict(df_cluster2)
```



```

# Features
X = df_cluster2.drop(columns="cluster")

# PCA
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X)

# Dataframe PCA
df_pca = pd.DataFrame(X_pca, columns=["PC1", "PC2"])
df_pca["cluster"] = df_cluster2["cluster"].values

# Plot
plt.figure(figsize=(8,6))

colors = {0: "blue", 1: "green", 2: "orange"}

for c in [0, 1, 2]: # <-- evita NaN direttamente
    subset = df_pca[df_pca["cluster"] == c]
    plt.scatter(
        subset["PC1"],
        subset["PC2"],
        s=10,
        alpha=0.5,
        label=f"Cluster {c}",
        color=colors[c]
    )

# Centroids
centroids_pca = pca.transform(kmeans_demo.cluster_centers_)

plt.scatter(
    centroids_pca[:, 0],
    centroids_pca[:, 1],
    marker='X',
    s=200,
    color='black',
    label='Centroids'
)

plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("PCA Projection of Demographic Clusters")

plt.legend()
plt.show()

```

The PCA projection confirms the existence of three distinct demographic segments:

- Cluster 1 represents high-income, urban donors, clearly separated along the first principal component.
- Cluster 2 captures economically vulnerable donors, strongly differentiated along the second component, driven by housing instability.
- Cluster 0 lies between these extremes, representing a more stable and moderate socio-economic group.

Although some overlap exists, the clusters exhibit clear structural separation, supporting the validity of the segmentation.

```
X = df_cluster1.copy()
```

```

kmeans_beh = KMeans(n_clusters=3, random_state=42)

df_cluster1["behavior_cluster"] = kmeans_beh.fit_predict(X)

```

```
print(df_cluster1["behavior_cluster"].value_counts(normalize=True))
```

```
df_cluster1.columns
```

```
df_cluster2.columns
```

```

# Rename clusters for clarity
df_cluster1 = df_cluster1.rename(columns={"cluster": "behavior_cluster"})
df_cluster2 = df_cluster2.rename(columns={"cluster": "demo_cluster"})

```

```

# Merge on index
df_cross = df_cluster1.join(df_cluster2["demo_cluster"])

```

```
df_cross
```

```

cross_tab = pd.crosstab(
    df_cross["behavior_cluster"],

```

```
df_cross["demo_cluster"],  
normalize="index"  
).round(3)  
  
cross_tab
```

The cross-analysis between behavioral and demographic clusters shows no strong one-to-one correspondence between the two segmentations.

Each behavioral cluster is composed of a mix of demographic profiles, indicating that donor behavior is not solely driven by socio-economic characteristics.

This suggests that individuals with similar economic backgrounds can exhibit different donation behaviors, and conversely, donors with diverse demographic profiles may behave similarly.

The two clustering perspectives are therefore complementary and provide a richer segmentation when combined